The Awesome Print Example

The awesome_print gem prints Ruby objects on the screen with indentation, color, and other niceties:

object_model/awesome_print_example.rb
```
require "awesome_print"

local_time = {:city => "Rome", :now => Time.now }
ap local_time, :indent => 2
```

This produces:

```
{
  :city => "Rome",
  :now => 2013-11-30 12:51:03 +0100
}
```

You can call ap from anywhere because it's a *Kernel Method (32)*, which you can verify by peeking into Awesome Print's source code:

gems/awesome_print-1.1.0/lib/awesome_print/core_ext/kernel.rb
```
module Kernel
  def ap(object, options = {})
    # ...
  end
end
```

After this foray into Ruby modules and the Kernel, you can finally learn how Ruby executes methods after finding them.

Method Execution

When you call a method, Ruby does two things: first, it finds the method, and second, it executes the method. Up to now, you focused on the finding part. Now you can finally look at the execution part.

Imagine being the Ruby interpreter. Somebody called a method named, say, my_method. You found the method by going one step to the right, then up, and it looks like this:

```
def my_method
  temp = @x + 1
  my_other_method(temp)
end
```

To execute this method, you need to answer two questions. First, what object does the instance variable @x belong to? And second, what object should you call my_other_method on?